

WaveBench Studio

Technical Architecture & Execution Plan

Graphical Programming Environment — PASS Technical Team Assignment

Candidate	[Your Name]
-----------	-------------

Department / College	[Fill in]
-------------------------	-----------

Prepared	July 12, 2026 (revised: frontend swapped to React)
----------	--

Target Submission	July 21, 2026 (<i>see deadline note below</i>)
-------------------	--

Stack	React + TypeScript (frontend) · Java 17 (WebSocket gateway) · C++17 (simulation engine) · VS Code
-------	--

Revision note: Per your message, the frontend has been switched from JavaFX to React, as explicitly permitted in the recruiter's email. Java is preserved as a required **WebSocket-to-TCP gateway** — not a token inclusion, but a technical necessity, since browsers cannot open raw TCP sockets. This keeps the submission genuinely "integrable with Java code" as instructed, while React handles the actual UI.

⚠ Deadline Discrepancy — Confirm This First

Two different dates appear in your materials:

- **Assignment PDF:** *"Deadline for Assignment: 28 July 2026"*
- **Recruiter email:** *"on or before 21 July 2026"*

This plan treats **21 July** as the real deadline (a 9-day sprint from today), with the extra week as buffer if the PDF date is authoritative instead. Send a one-line confirmation email to PASS Technical Team before investing 9 days of work.

Contents

1. Concept — What Makes This Submission Stand Out
2. High-Level Architecture
3. Tech Stack
4. Block-to-Code Mapping

5. Communication Protocol
 6. Class & Module Design
 7. PDF Annotation Compliance Map
 8. Repository Structure
 9. Day-by-Day Build Plan
 10. One-Click Run Setup (VS Code)
 11. Innovation Checklist
 12. Risks & Mitigations
 13. Final Report Outline
 14. Submission Checklist
-

1. Concept — What Makes This Submission Stand Out

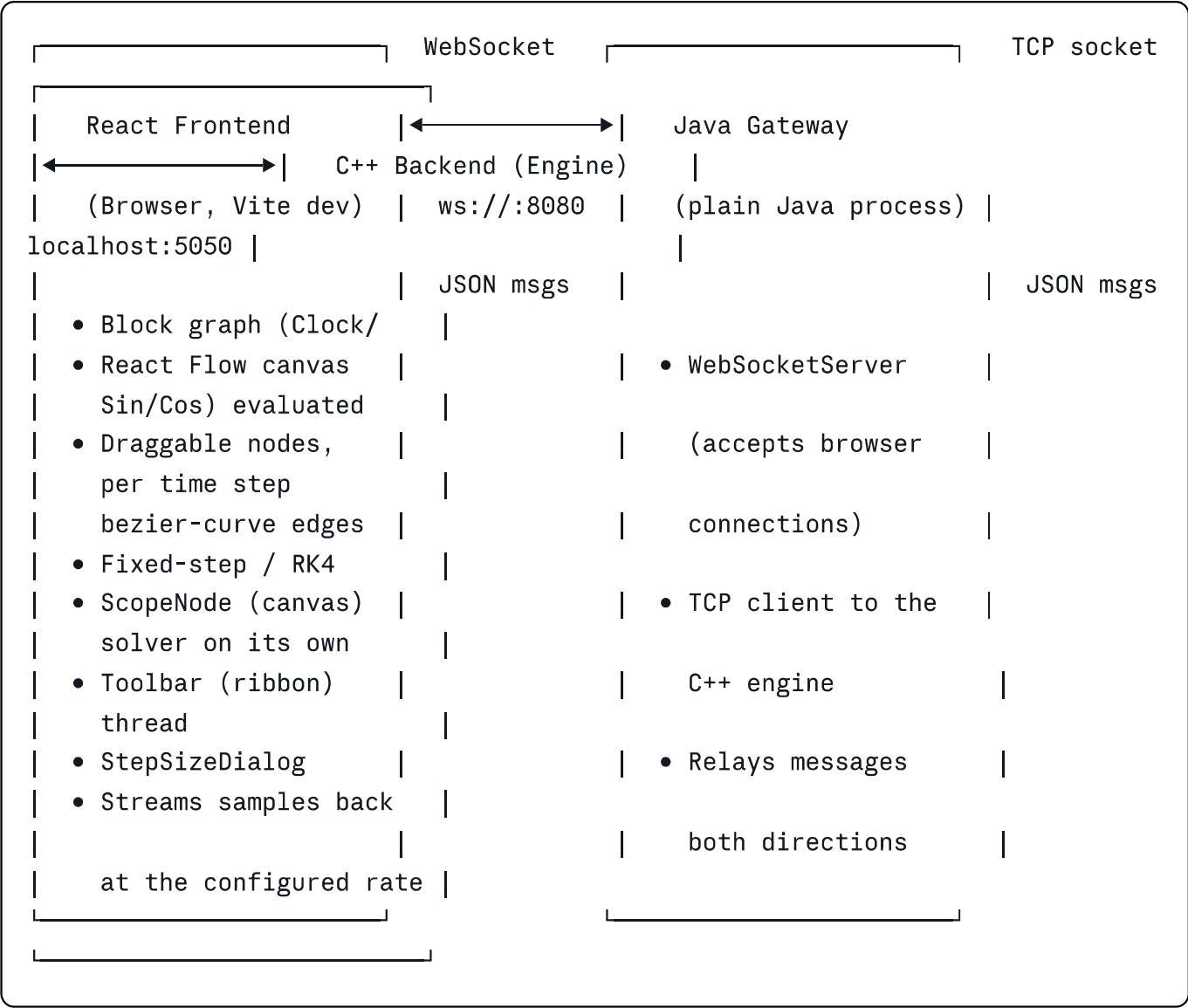
Most candidates who pivot to React will do the minimum: a React app that renders four boxes and computes $\sin/\cos$ directly in JavaScript, with C++ nowhere near the actual data path. That satisfies "use React" but quietly drops the "integrable with Java code" requirement — an easy way to lose points without realizing it.

WaveBench Studio instead keeps the same three-layer separation of concerns from the original plan, just with a browser-friendly frontend:

- **Frontend (React)** owns rendering, layout, drag-and-drop, and interaction. Built on **React Flow**, a library purpose-built for exactly this kind of node/edge diagram editor — it handles draggable nodes and connectable bezier-curve wires natively, which is the single most time-consuming part of a from-scratch JavaFX build.
- **Java Gateway** is the real, load-bearing reason Java stays in the project: it's the only thing standing between the browser (which only speaks HTTP/WebSocket) and the C++ engine (which speaks raw TCP). It relays commands one way and streamed samples the other.
- **C++ Backend** is completely unchanged from the original plan — same **Block** graph, same solver loop, same TCP/JSON server. React doesn't care what's computing the numbers; it just draws what it receives.

This is the pitch to lead with in your report: *"Java isn't included to check a box — it's the architectural piece that makes a browser-based frontend possible at all, given a native C++ simulation core."*

2. High-Level Architecture



Why the Java gateway is necessary, not decorative

Browsers cannot open a raw TCP socket to `localhost:5050` — the Fetch/WebSocket APIs are the only network primitives available to browser JS. So *something* has to translate. Two options:

	Java WebSocket gateway (recommended)	C++ speaks WebSocket directly
Satisfies "integrable with Java code"	Yes — Java is genuinely load-bearing	No — Java would be absent entirely
Implementation effort	Low — one small Java class using the <code>Java-WebSocket</code> library	Higher — hand-rolling the WebSocket handshake/framing in C++
Debuggability	Each hop testable independently (raw TCP to C++, raw WS to Java)	Single hop, but harder to isolate failures

Fallback only if truly time-constrained: skip the gateway and have C++ speak a minimal WebSocket handshake directly. This drops Java from the data path entirely, so only use it as a last resort — flag clearly in your report if you do, since it no longer meets the letter of the email's instruction.

3. Tech Stack

Layer	Technology	Notes
Frontend framework	React 18 + TypeScript	Scaffolded via Vite (<code>npm create vite@latest</code>) for a fast dev server and simple single build step
Diagram canvas	React Flow (<code>@xyflow/react</code>)	Gives draggable nodes and connectable bezier edges out of the box — replaces the custom <code>Pane</code> / <code>CubicCurve</code> work from the JavaFX plan almost entirely
Scope rendering	Native <code><canvas></code> + <code>useRef</code> , imperative draw calls	Deliberately <i>not</i> React state per sample — redrawing via React re-renders at 60fps would thrash the DOM; draw directly on the canvas context instead
Frontend transport	Native browser <code>WebSocket</code> API	No extra library needed for this scale
Bridge language	Java 17	The required "integrable with Java" piece
Bridge WebSocket lib	<code>Java-WebSocket</code> (<code>org.java-websocket</code>)	Single Maven dependency, minimal boilerplate <code>WebSocketServer</code> subclass
Bridge build	Maven	<code>mvn -f gateway/pom.xml exec:java</code> as the single gateway launch command
Backend language	C++17	Unchanged from original plan
Compiler	MinGW-w64 <code>g++</code>	As specified in the PDF

Layer	Technology	Notes
Backend networking	Winsock2 (<code><winsock2.h></code> , <code>-lws2_32</code>)	Unchanged
JSON (all three layers)	Minimal hand-rolled encode/decode, or <code>JSON.parse</code> / <code>JSON.stringify</code> natively in React/TS	The message schema (Section 5) is simple enough that no heavy dependency is needed anywhere
IDE	VS Code	<code>Extension Pack for Java</code> , <code>C/C++</code> (<code>ms-vscode.cpptools</code>), and the built-in TS/JS tooling all in one workspace
Automation	<code>.vscode/tasks.json</code>	Now sequences three processes — see Section 10

4. Block-to-Code Mapping

Required Block	Backend Class (C++)	Frontend Node (React)	Behavior
Clock (Signal Generator)	<code>ClockBlock : Block</code>	<code>ClockNode</code> (React Flow custom node)	<code>compute(t) = t</code> — master time reference
Sine (Function Processor)	<code>SineBlock : Block</code>	<code>SineNode</code>	<code>compute(t) = sin(source->compute(t))</code>
Cosine (Function Processor)	<code>CosineBlock : Block</code>	<code>CosineNode</code>	<code>compute(t) = cos(source->compute(t))</code>
Scope (Visualizer)	(<i>sink, aggregated server-side</i>)	<code>ScopeNode</code> (wraps a <code><canvas></code>)	Receives streamed <code>{t, sin, cos}</code> samples, plots both traces

Routing (fixed per spec): Clock → Sine, Clock → Cosine, Sine → Scope, Cosine → Scope — defined as a static `edges` array in React Flow (Section 6). Block *positions* are draggable by default; re-wiring the graph itself isn't required by the PDF, so don't over-scope it.

5. Communication Protocol

Same message schema as before — now traveling over **two hops** instead of one: browser ⇄

Java over WebSocket, Java $\rightleftharpoons$ C++ over TCP. The Java gateway does no transformation, just relays the raw JSON string in both directions.

Client (React) $\rightarrow$ Gateway (Java) $\rightarrow$ Engine (C++):

```
json

{"type": "config", "stepSize": 0.02, "solver": "RK4"}
{"type": "start"}
{"type": "setStepSize", "value": 0.05}
{"type": "stop"}
```

Engine (C++) $\rightarrow$ Gateway (Java) $\rightarrow$ Client (React), streamed continuously:

```
json

{"type": "sample", "t": 0.42, "sin": 0.4078, "cos": 0.9130}
```

Design notes:

- The C++ engine still runs its simulation loop independently of network speed, pushing samples into a thread-safe queue drained by a sender thread — unchanged from the original design.
- The Java gateway should **connect to the C++ engine first**, then start accepting browser WebSocket connections — otherwise a browser could connect before there's anything to relay. A simple retry-connect loop on gateway startup handles this.
- React's `ScopeNode` keeps only the last N samples (e.g. 500) in a ref-backed ring buffer, not React state, to avoid re-render storms.

6. Class & Module Design

C++ Backend — unchanged from the original plan:

```
cpp
```

```
// Block.hpp – abstract base for all simulation blocks
class Block {
public:
    virtual double compute(double t) = 0;
    virtual ~Block() = default;
};

class SineBlock : public Block {
    Block* source;
public:
    explicit SineBlock(Block* src) : source(src) {}
    double compute(double t) override { return std::sin(source->compute(t)); }
};
```

Java Gateway — the new, load-bearing piece:

```
java

// GatewayServer.java – accepts browser WebSocket connections, relays to/from C
public class GatewayServer extends WebSocketServer {
    private final EngineClient engine;

    public GatewayServer(int port, EngineClient engine) {
        super(new InetSocketAddress(port));
        this.engine = engine;
    }

    @Override
    public void onMessage(WebSocket conn, String message) {
        engine.send(message);        // forward browser command to C++
    }

    public void broadcastSample(String json) {
        broadcast(json);              // push a C++ sample to all connected brow
    }
}
```

```
java
```

```
// EngineClient.java – plain TCP client to the C++ SimulationEngine
public class EngineClient {
    private PrintWriter out;

    public void connect(String host, int port, Consumer<String> onSample) throws
        SocketException {
        Socket socket = new Socket(host, port);
        out = new PrintWriter(socket.getOutputStream(), true);
        new Thread(() -> {
            try (var in = new BufferedReader(new InputStreamReader(socket.getIn
                String line;
                while ((line = in.readLine()) != null) onSample.accept(line);
            } catch (IOException ignored) {}
        }).start();
    }

    public void send(String json) { out.println(json); }
}
```

React Frontend:

```
tsx

// useSimulationSocket.ts – WebSocket hook, exposes samples + command senders
export function useSimulationSocket() {
    const wsRef = useRef<WebSocket>();
    const samplesRef = useRef<Sample[]>([]);

    useEffect(() => {
        const ws = new WebSocket("ws://localhost:8080");
        ws.onmessage = (e) => {
            const msg = JSON.parse(e.data);
            if (msg.type === "sample") {
                samplesRef.current = [...samplesRef.current.slice(-499), msg];
            }
        };
        wsRef.current = ws;
        return () => ws.close();
    }, []);

    const send = (obj: object) => wsRef.current?.send(JSON.stringify(obj));
    return { samplesRef, send };
}
```

tsx


```
// App.tsx — the fixed 4-block diagram, wired once at startup
const nodeTypes = { clock: ClockNode, sine: SineNode, cosine: CosineNode, scope

const initialNodes = [
  { id: "clock", type: "clock", position: { x: 50, y: 150 }, data: {} },
  { id: "sine", type: "sine", position: { x: 300, y: 80 }, data: {} },
  { id: "cosine", type: "cosine", position: { x: 300, y: 220 }, data: {} },
  { id: "scope", type: "scope", position: { x: 560, y: 150 }, data: {} },
];

const initialEdges = [
  { id: "e1", source: "clock", target: "sine", animated: true },
  { id: "e2", source: "clock", target: "cosine", animated: true },
  { id: "e3", source: "sine", target: "scope", animated: true },
  { id: "e4", source: "cosine", target: "scope", animated: true },
];
```

Note the `animated: true` on each edge — React Flow renders a moving-dash "data flowing" animation on the wire for free, which is the exact signal-flow visualization that took custom work in the JavaFX plan (see Section 11).

Other modules: `ToolbarView.tsx` (ribbon, see Section 7), `StepSizeDialog.tsx` (modal popup — the red-annotation innovation), `ScopeCanvas.tsx` (imperative `<canvas>` drawing inside `ScopeNode`).

`<a id="7-compliance"></a>`

7. PDF Annotation Compliance Map

Annotation (from reference screenshots)	Requirement	How This Plan Satisfies It
● Full top ribbon — tabs (Simulation / Debug / Modeling / Format / Apps / Scope) + all four toolbar groups underneath (File: New/Open/Print · Library: Library Browser · Prepare: Log Signals/Add Viewer/Signal Table · Simulate: Stop Time/Normal mode/Fast Restart/Step Back/Run)	Keep functionality as-is	Rendered as a <code>Toolbar</code> React component reproducing the tabs and all four groups; free to reposition — plan keeps it as a top ribbon but simplified visually rather than moving it elsewhere
● Window/tab title — <i>"untitled* - Simulink academic use"</i>	Keep the naming convention	Browser tab title (<code>document.title</code>) set to <code>"untitled* - WaveBench Studio (academic use)"</code>

Annotation (from reference screenshots)	Requirement	How This Plan Satisfies It
Step-size definition area, bottom-right — "a new pop-up window should appear here"	Free to innovate	StepSizeDialog — a React modal with a step-size slider, solver selector (Euler / RK4), and a small live preview plot

Not annotated (optional, not a hard requirement): the reference screenshot also shows a left "Model Browser" rail with zoom/fit/select/text/image/shape tool icons. This isn't marked green or red in the PDF, so it's a nice-to-have visual nod to Simulink's aesthetic (Section 11), not something you need to reproduce for full marks.

8. Repository Structure

```
wavebench-studio/
├── backend/                                (C++ engine - unchanged)
│   ├── src/
│   │   ├── main.cpp
│   │   ├── Block.hpp
│   │   ├── SimulationEngine.hpp / .cpp
│   │   └── ServerSession.hpp / .cpp
├── gateway/                               (NEW: Java WebSocket ⇌ TCP bridge)
│   ├── pom.xml
│   └── src/main/java/com/wavebench/gateway/
│       ├── Main.java
│       ├── GatewayServer.java
│       └── EngineClient.java
├── frontend/                             (NEW: React app, replaces the JavaFX
project)
│   ├── package.json
│   ├── vite.config.ts
│   └── src/
│       ├── App.tsx
│       ├── components/Toolbar.tsx, StepSizeDialog.tsx
│       ├── nodes/ClockNode.tsx, SineNode.tsx, CosineNode.tsx, ScopeNode.tsx
│       ├── canvas/ScopeCanvas.tsx
│       └── hooks/useSimulationSocket.ts
├── .vscode/
│   └── tasks.json
└── docs/
```


9. Day-by-Day Build Plan (9-day sprint, targeting 21 July)

Day	Date	Focus	Deliverable
1	Jul 13	Environment + repo scaffold	C++, Java (Maven), and React (Vite) projects each build/run a "hello" independently
2	Jul 14	Backend core engine	<code>Block</code> hierarchy + <code>SimulationEngine</code> printing samples to console
3	Jul 15	Backend TCP server + gateway TCP client	C++ streams JSON over TCP; Java <code>EngineClient</code> connects and logs incoming samples
4	Jul 16	Gateway WebSocket server	Browser-side testable via a raw WS client (e.g. <code>websocat ws://localhost:8080</code>)
5	Jul 17	React scaffold + React Flow canvas	Toolbar (green-compliant) + 4 nodes rendered, draggable, wired with the fixed edges
6	Jul 18	Live integration	<code>useSimulationSocket</code> connected; <code>ScopeCanvas</code> plotting live sin/cos from the real pipeline
7	Jul 19	Step-size popup	<code>StepSizeDialog</code> modal, wired to <code>setStepSize</code> / <code>config</code> messages
8	Jul 20	Polish + innovation extras	Theming, FPS/latency HUD, any Section 11 extras
9	Jul 21	Packaging + report + submit	<code>tasks.json</code> one-click flow verified across all three processes; README + report done; email sent

10. One-Click Run Setup (VS Code)

Three processes now need to start in order: **C++ engine** → **Java gateway** → **React dev server**. `.vscode/tasks.json` handles the sequencing:

```
jsonc
```

```

{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Build Backend",
      "type": "shell",
      "command": "g++ -std=c++17 -O2 backend/src/*.cpp -lws2_32 -o backend/build",
    },
    {
      "label": "Run Backend",
      "type": "shell",
      "command": "backend/build/wavebench_engine.exe",
      "isBackground": true,
      "dependsOn": "Build Backend"
    },
    {
      "label": "Run Gateway",
      "type": "shell",
      "command": "mvn -f gateway/pom.xml exec:java",
      "isBackground": true,
      "dependsOn": "Run Backend"
    },
    {
      "label": "Run Frontend",
      "type": "shell",
      "command": "npm --prefix frontend run dev",
      "dependsOn": "Run Gateway"
    },
    {
      "label": "Run WaveBench (One Click)",
      "dependsOn": ["Run Backend", "Run Gateway", "Run Frontend"],
      "dependsOrder": "sequence",
      "group": { "kind": "build", "isDefault": true }
    }
  ]
}

```

Reviewer opens the folder in VS Code, hits **Ctrl+Shift+B**, and all three layers come up in order, ending with Vite printing a `localhost:5173` link to open in the browser. Document this clearly in the README — a reviewer shouldn't need to open three manual terminals.

11. Innovation Checklist

- ☐ **Solver-selection popup** (core innovation target): step-size slider + Euler/RK4 dropdown + live preview curve
- ☐ **Signal-flow animation** — essentially free via React Flow's `animated: true` edge option; worth calling out explicitly in your report as a deliberate touch, not just a default
- ☐ Dark, Simulink-inspired theme (CSS variables / Tailwind)
- ☐ FPS / gateway-latency HUD overlay
- ☐ Save/load diagram layout as JSON (React Flow exposes node positions directly)
- ☐ Export scope data to CSV
- ☐ Snap-to-grid dragging (built into React Flow via a prop)
- ☐ Keyboard shortcuts (Delete, Ctrl+Z undo)

Pick 2-3 — a couple of polished extras beat a pile of half-finished ones.

12. Risks & Mitigations

Risk	Mitigation
Gateway starts before C++ engine is ready	Retry-connect loop in <code>EngineClient.connect()</code> on gateway startup
Browser connects to gateway before engine link is established	Gateway can queue/reject early <code>onMessage</code> calls until <code>EngineClient</code> reports connected
Three-process orchestration failing silently	Each process logs clearly to its own terminal panel in VS Code; test each hop independently before wiring end-to-end (Day 3-4)
<code>ScopeNode</code> re-rendering too often and dropping frames	Draw on <code><canvas></code> imperatively via a ref, never store per-sample data in React state
MinGW/Winsock linking errors	Validate a minimal hello-socket program on Day 1
Deadline ambiguity (21 vs 28 July)	Email the recruiter today; plan already targets the earlier date

13. Final Report Outline

- Overview & problem statement

2. Architecture diagram + explanation of the three-layer split (**lead with why Java's role is technically necessary, not just compliant**)
3. Tech stack & justification (React Flow's fit for node/edge editors, why a Java bridge rather than C++ speaking WebSocket directly)
4. Feature walkthrough with screenshots or a short GIF
5. How to run — the one-click steps
6. Design decisions & trade-offs
7. Innovations added (Section 11 items you actually built)
8. Challenges faced & how you solved them
9. Future improvements
10. Appendix: class diagram, protocol spec (Section 5 reusable directly)

Happy to generate this as a polished Word doc (`.docx`) once the build is done and you have real screenshots — just ask.

14. Submission Checklist

- ☐ Code compiles/runs from a clean clone via one command (Ctrl+Shift+B)
- ☐ All three processes (C++, Java, React) start in the correct order and connect
- ☐ All 4 blocks present, correctly wired, individually draggable
- ☐ Scope shows live, simultaneous sine and cosine traces
- ☐ Step-size popup functional (solver + step size)
- ☐ Green toolbar elements and title-naming convention retained
- ☐ README explains the one-click run process and the 3-layer architecture
- ☐ `report.docx` included in `docs/`, explicitly explaining Java's role
- ☐ Repo zipped or GitHub link ready
- ☐ Email sent to **sivaraman.s@alumni.iitm.ac.in** with name, department, college, and mobile number
- ☐ Actual deadline (21 vs 28 July) confirmed with the recruiter